ARCH LINUX 分册教材

第一册

Arch Linux 与它的社区

发行版哲学、项目结构、社区协作与本地化

依据 Arch Linux 中文维基全量条目编写

共 9 册 · 997 篇 · 安装 / 系统 / 网络 / 硬件 / 桌面 / 软件 / 开发 / 安全

目　录

第 1 章　Linux 与 Arch Linux

第 2 章　终端与命令行

第 3 章　文件系统与权限

第 4 章　软件包与依赖

第 5 章　系统与服务

第 6 章　学习方法与故障排查

Linux 与 Arch Linux

一台电脑刚买回来的时候，那些零件本身什么都不会做。处理器在等指令，内存在等数据，硬盘在等读写请求。要把这些冷冰冰的东西变成你能用的机器，中间必须有一层软件来统一调度——这一层就是操作系统（operating system，简称 OS）。

操作系统在做什么

把一台电脑想成一家餐厅。硬件是厨房里的灶台、冷库、备餐台；你打开的各种程序，是来点菜的客人。厨房不能谁想进就进、想拿什么就拿什么，否则非打起来不可。所以餐厅需要一个后厨总管：谁先用灶、食材从哪儿出、做好的菜端到哪张桌，都由他统一安排。

操作系统干的就是这件事。它不直接替你写文档、看视频，但它决定哪个程序此刻能用处理器、能分到多少内存、能读写哪块硬盘、能不能联网。任何程序想碰硬件，都得向操作系统提申请，由它代办。这个统一受理申请的机制，叫系统调用（system call）。

操作系统内部又分成两层，这个分工值得先记住。

最里面那层叫内核（kernel）。它直接和硬件打交道，掌管处理器时间怎么分配、内存怎么划分、磁盘和网卡怎么驱动。内核的权限极高，一旦它出错，整台机器可能直接停摆。所以内核被刻意设计得小而克制，只做最必要的调度与隔离。

内核之外的所有东西，统称用户态（user space）。你看到的桌面、浏览器、编辑器、终端窗口，全都活在这一层。它们没有直接操作硬件的权力，想做任何出格的事，都得回头请内核帮忙。这种"内核守门、用户态干活"的结构，好处是隔离：一个浏览器崩了，通常不会把整台机器一起带走。

一句话概括：内核是权力的中心，用户态是生活的地方。

发行版：同一台发动机，不同的整车

严格说，Linux 只是一个内核的名字。它由 Linus Torvalds 在 1991 年发起，今天由全球开发者共同维护。但只有内核，电脑是没法用的——你还需要命令行工具、编译器、图形界面、驱动程序、网络配置手段，以及一套能把它们装上、升级、卸载的机制。

内核之外的这些东西，绝大部分来自 GNU 计划和其他开源项目。所以严谨的说法是 GNU/Linux，日常口语里就简称 Linux。

问题在于，把内核和成百上千个软件拼成一套能跑的系统，是一件极其繁琐的活。每个软件的版本要互相匹配，配置文件要写对，出了问题得有人修。于是就有了发行版（distribution，简称 distro）。

可以用汽车打比方。发动机、变速箱、底盘这些东西，都有专门的厂商在做，它们是上游（upstream）——原始开发者，按自己的节奏发布新版本。发行版则是整车厂：选哪款发动机、配什么变速箱、内饰怎么布置、售后网点开在哪儿，都由它决定。同一台发动机，可以装出家用轿车，也可以装出越野车。

发行版的核心资产通常有三样：

- 一个软件仓库（repository），即一台服务器，存着几千个已经编译好、彼此兼容的软件包；
- 一个包管理器（package manager），负责从仓库取回软件、装上、记下它依赖哪些别的软件，以后统一升级或卸载；
- 一套默认配置与一份安装说明，让这套东西能开箱就用。

这里的"依赖"（dependency）值得多解释一句。软件很少孤立存在，一个播放器可能要用到某个解码库，那个库又依赖某个基础运行库。就像装一盏吊灯，除了灯本身，还得有膨胀螺丝和电线。包管理器的工作，就是自动把这些螺丝和电线一并备齐，并在升级时保证它们仍然对得上。

不同发行版在这三样上的取舍不同，于是长成了不同的性格。比较主流的几大家族有：Debian 及其衍生的 Ubuntu、Red Hat 系的 Fedora、openSUSE，以及本章要讲的 Arch Linux。

Arch 在其中的位置

Arch Linux 于 2002 年由 Judd Vinet 创建，如今由 Arch 社区自身维护。

如果继续用装修打比方，多数发行版交付的是精装房：地板铺好、家电摆好，拎包入住。Arch 交付的是毛坯房，外加一套质量很好的建材和一本极其详尽的施工手册。装成什么样，是你的自由，也是你的责任。

这个定位带来几个直接后果。Arch 的安装过程不会替你做太多决定，分区、选装哪些组件、怎么写配置，都得自己来。装完之后系统里只有你点名要的东西——没有预装的办公套件，也没有替你搭好的桌面环境。想要什么，自己装。

对新手来说这听着像刁难，但对另一部分人来说这正是价值所在：系统里每一层你都认识，出了问题你能一路查下去，而不是对着一堆来路不明的预装软件猜。

三条设计原则

Arch 官方把自己的取舍归纳为一组原则，通常称为 Arch 之道（The Arch Way）。其中对使用者影响最直接的是三条。

简洁（simplicity）。 这里的简洁不是"界面好看、步骤少"，而是"系统里没有多余的中间层"。Arch 不会给你一个图形化向导把细节藏起来，也不会在你不知情时替你改配置文件。它更愿意只提供一个清晰的机制，让你自己决定怎么用。代价是你要多懂一点，收益是系统里没有你解释不了的东西。

现代（modernity）。 Arch 紧跟上游，软件包里装的通常就是上游发布的最新稳定版本，而不是被发行版冻结许久、再打上一堆补丁的旧版本。好处是新硬件、新特性来得快；代价是上游的 bug 也会更快到你面前。

务实（pragmatism）。 原则归原则，能解决问题才算数。Arch 不会为了某种理念上的纯粹，硬把可用的东西拒之门外；该用的补丁会用，该妥协的地方会妥协。它追求的是"这套东西对开发者有用"，而不是"这套东西在理论上完美"。

三条合起来其实是一句话：Arch 把决定权交给你，同时不承诺替你兜底。

滚动更新：没有"大版本"这回事

多数发行版采用固定版本（fixed release）模式。它们定期冻结一次软件版本，打包成一个大版本发布，比如某个年份某个编号。两次发布之间，除安全修复外软件版本基本不动。你在用的是一份被定格的快照，好处是稳定可预期，代价是半年或一年内拿不到新东西。

Arch 不这么干。它采用滚动更新（rolling release）：没有版本号，没有"Arch 25"这样的叫法，只有一个持续流动的软件仓库。你随时同步仓库，就能拿到当下最新的整体状态。系统永远处于"当前"，而不是"某个已发布的版本"。

滚动更新省掉了不少麻烦：不存在"跨大版本升级"这种需要备份、重装、祈祷的大手术；也不存在"文档写的是新版、你手上是旧版"的错位——Arch 的文档与仓库状态始终同步。

但代价是实打实的，而且必须提前知道。

第一，没有稳定期的缓冲。 上游发布新版本，经过打包和测试就会进入仓库。你没有"等半年让别人先踩掉问题"的选项，某些更新确实会带来回归问题。

第二，更新是日常动作，不是年度事件。 固定版本用户一年更新一次，Arch 用户需要经常同步。长期不更新之后再一次性同步，风险会显著上升。

第三，绝对不能做部分更新。 这一点最容易被新手忽视。Arch 的仓库是一个整体，各软件包之间的依赖关系按当前仓库状态对齐。只升级其中某一个包，很可能让整台机器陷入互相不匹配的状态。正确的做法永远是整仓库一次性更新：

```
# pacman -Syu
```

`pacman` 是 Arch 的包管理器，`-Syu` 的意思是同步仓库信息并把系统里所有包一起升级。请把这条命令当成整体，不要拆开只更新单个包。

第四，你需要读公告。 少数更新需要人工干预，比如某个配置文件换了格式、某个服务改了默认行为。Arch 会把这些写在更新公告里。更新前扫一眼公告，是使用 Arch 的基本习惯。

学 Arch 需要什么心理准备

最后说点实际的。

你会自己装系统。 没有人替你一路点"下一步"。分区表写错、引导程序装错位置，都是新手会经历的事。这不是天赋问题，是熟练度问题。

你会读大量英文文档。 Arch Wiki 是这个发行版最重要的资产，写得极其详尽，且紧跟当前状态。它不假设你懂，但也不哄你。把它当第一手资料用，是这个社区默认的协作方式。

你会把系统弄坏。 认真说，这不是失败。Arch 不阻止你做危险操作，也不在你误操作时跳出来挡住你。设计上的假设是：使用者知道自己在做什么，并愿意为自己的操作负责。弄坏了就查文档、进恢复环境、修回来——这个过程本身就是最好的学习。

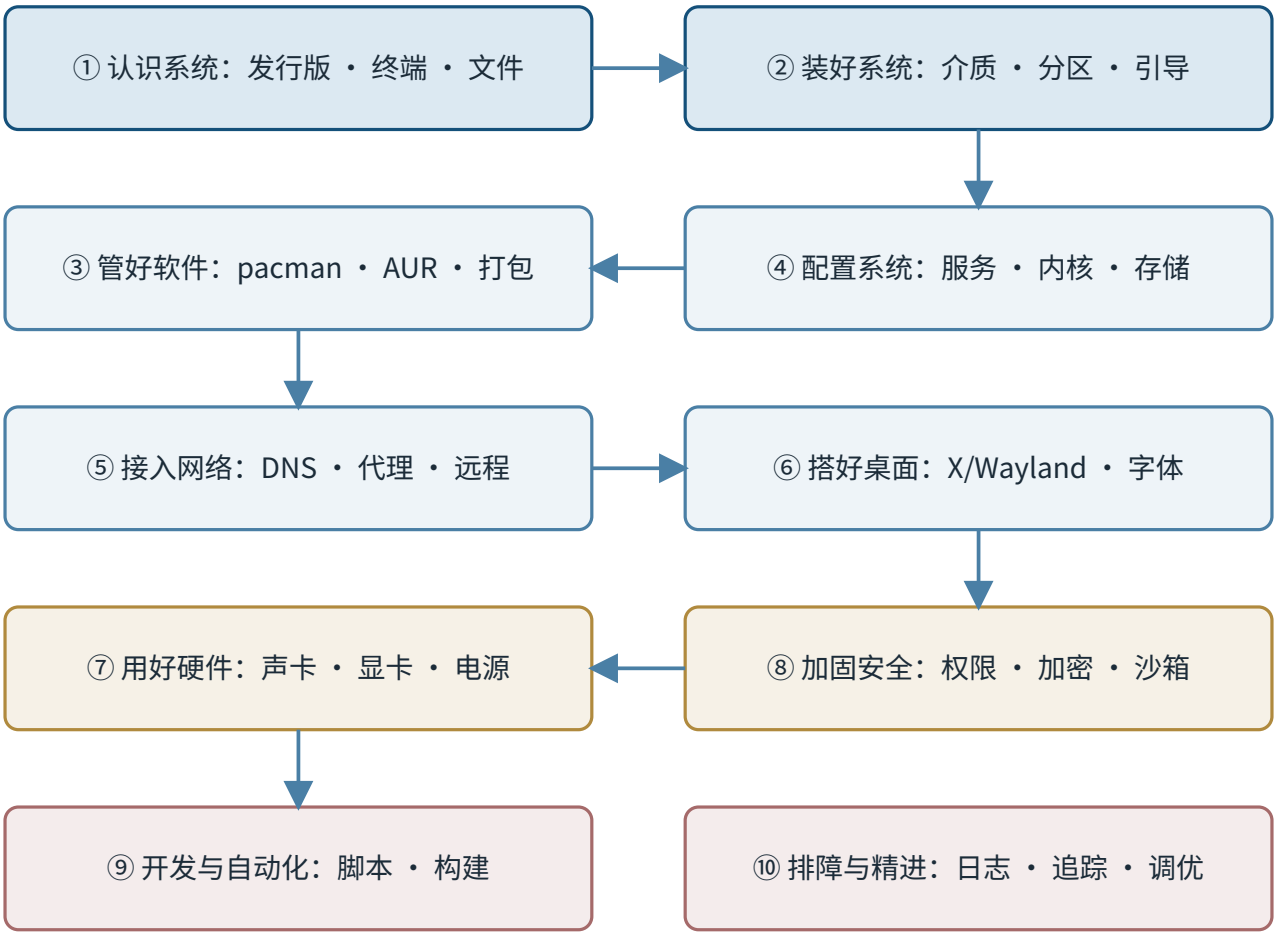
你不需要先成为程序员。 零基础完全可以开始，但你需要愿意动手、愿意追问"这一步到底在干什么"，而不是只求一条能照抄的命令。

不要把 Arch 当成"进阶版 Ubuntu"。 它不只是更难装，它是另一种契约：发行版少替你做决定，你多承担一部分责任。接受这个契约，用起来会很痛快；不接受，就会觉得处处别扭。

这套契约值不值得，取决于你想从这台机器上得到什么。如果只是想装好就能用，别的发行版更省心；如果你想弄明白一台电脑到底怎么跑起来，Arch 是一条很直接的路——因为它几乎不藏东西。

学习路径

从零基础到能独立维护系统



十步学习路径，对应本教材九册的推进顺序

第 2 章

终端与命令行

为什么 Arch 离不开终端

大部分发行版在第一次开机时就把桌面摆好了：图标、菜单、设置面板，每件事都有对应的按钮。Arch 的做法不同，它给你的是一套工具，而不是一件成品。好处是你能完全决定系统长什么样，代价是很多步骤没有现成的按钮可点。

安装 Arch 本身，官方给出的流程就是一行行命令；系统装好之后，更新靠的也是一条命令，而不是某个“检查更新”的图标。这不是说 Arch 用不了图形界面——它的桌面环境和其他发行版一样齐全。只是当系统出问题、需要看清它到底做了什么的时候，图形界面会把细节藏起来，命令行不会。

所以这一章的目标不是让你背下命令，而是让你在面对一个空白的终端窗口时，知道自己在敲什么。

终端里住着一个 shell

先分清两样东西，它们经常被混为一谈。

终端（terminal）是你看到的那个窗口。它本身几乎不做决定，只负责显示文字、接收按键。窗口里真正干活的程序叫 shell，中文常译作“壳”——包在操作系统内核外面的一层壳。

shell 的职责是一个循环：打印提示符，等你敲一行字；把这一行读进来，解析成“要运行哪个程序、带哪些材料”；把程序跑起来；等它结束，把结果交回屏幕；然后接着等下一行。这个循环一直重复，直到你关掉窗口。

Arch 默认的 shell 是 bash。你敲下的每个字都先经过它，由它决定把工作交给谁。也正因为有这一层，命令行里能做的事情，比任何一个图形程序都要多。

一条命令由什么组成

看这一行：

```
$ ls -l /home
```

用空格切开，它是三个词。

第一个词 `ls` 是命令名（command），实际上是一个可执行文件的名字。shell 拿到它之后，会去几个约定的目录里找这个文件，找到就运行。

以减号开头的是选项（option），也叫标志（flag）。`-l` 表示用长格式列出，多显示权限、大小、时间这些信息。要留意：选项是每个命令自己定的规矩，不是 shell 的规则，同一个 `-l` 在这里是这个意思，换个命令可能就是别的意思。单字母选项写一根减号，完整单词的写法用两根，比如 `--all`。

剩下不带减号的部分叫参数（argument），说明这条命令作用在什么对象上。上面的 `/home` 就是一个参数。

命令名大多是从英文里截短的：`ls` 来自 list，`cd` 来自 change directory，`cp` 来自 copy，`mv` 来自 move，`rm` 来自 remove，`pwd` 是 print working directory 的缩写，`grep` 则出自编辑器里的 global regular expression print。知道出处，就不必死记。

空格在这件事上很重要。shell 靠空格切分词语，`ls -l` 是两个词，`ls-l` 会被当成一个不存在的命令名。多打一个空格通常无所谓，少打一个通常报错。

路径：东西放在哪儿

参数里最常见的是路径（path），也就是某个文件或目录的位置。

以斜杠 `/` 开头的叫绝对路径（absolute path），从文件系统的最顶层——根目录（root）——一路写下来。`/home` 的意思就是“根目录下的 home”。不论你现在站在哪里，绝对路径指向的都是同一个地方。

不以斜杠开头的叫相对路径（relative path），相对于你当前所在的位置。有两个符号要记住：`.` 表示当前目录，`..` 表示上一级目录。所以 `cd ..` 的意思是退回上一层。

还有个偷懒的写法：`~` 代表你的家目录（home directory），也就是你登录之后默认待的地方。它展开后是一串很长的绝对路径，但你不用记。

顺带一提，Linux 的路径分隔符是正斜杠 `/`，不是 Windows 习惯的反斜杠 `\`。这一条最容易让从 Windows 转过来的人出错。

管道与重定向

每个程序默认有两个出口和一个入口：出口是屏幕，分别叫标准输出（standard output）和标准错误（standard error）；入口是键盘，叫标准输入（standard input）。shell 可以改接这几条通道，这就是重定向（redirection）。

```
$ echo hello > note.txt
```

`>` 把本来该显示在屏幕上的内容送进文件。文件不存在会被创建，已经存在会被整个覆盖——覆盖是不打招呼的，所以对着一个正在用的文件敲 `>`，很容易把里面的内容

清空。想接在后面而不是覆盖，用 >>。反方向的 < 表示从文件读入，而不是等你敲键盘。

管道（pipe）的符号是竖线 |，它把前一条命令的输出直接变成后一条命令的输入，中间不必写成文件。

```
$ ls /etc | wc -l
```

前半句列出目录内容，后半句数行数，合起来就是“数一数这个目录下有多少个条目”。把小的程序首尾相接去解决一个大问题，是命令行最核心的思路：不指望有一个什么都干的程序，而是让很多只干一件事的程序串起来。

十个最常用的命令

先看它们长什么样，再逐条解释。

```
$ ls                        # 列出当前目录的内容
$ cd /etc                   # 切换到 /etc 目录
$ pwd                       # 打印当前所在目录
$ cat note.txt              # 把文件内容整体打印出来
$ less /etc/fstab           # 分页浏览文件
$ mkdir demo                # 新建一个目录
$ rm note.txt               # 删除文件
$ cp a.txt b.txt            # 复制
$ mv a.txt b.txt            # 移动或改名
$ grep root /etc/passwd     # 在一个文件里找出包含 root 的行
```

ls、cd、pwd 是“有什么、去哪儿、我在哪”的三件套。cd 不带参数会回到你的家目录，cd - 会回到上一次待过的目录。

cat 适合看一屏之内就能读完的短文件；less 适合看长文件，空格往下翻，方向键上下看，/ 后面跟关键词可以在页内搜索，按 q 退出。

mkdir -p 能一次性建出多层目录，中间几层不存在也不会报错。

rm 要格外小心：命令行里没有回收站，删掉就是删掉。删目录得加 -r（递归处理），复制目录同样要 cp -r。你在网上的教程里会频繁见到 rm -rf，它的意思是递归并且不再逐项确认——这条命令不区分对象，一次手滑就可能造成无法挽回的损失。不确定的时候，先用 ls 看清楚目录里是什么。

mv 同时承担“移动”和“改名”两件事，因为在文件系统里这本来就是同一种操作：换个名字，或者换个位置。

grep 在文件里按文本匹配，找某个配置项、翻某条日志都用它。

除了这十个，还有一个命令你迟早会天天用：pacman -Syu 用来同步软件源并升级整个系统。它属于包管理，后面有专门一章。

敲的时候不必全靠记忆

试一次就会发现，命令并不需要一次打对。

打到一半卡住，按一次 Tab 键：如果剩下的部分只有一种可能，shell 会替你补完；如果有多个候选，再按一次会把它们列出来。路径越长、文件名越怪，这个键越有用——按一下 Tab 往往比盯着屏幕回想可靠得多。

另一种省事的办法是翻历史。上下方向键可以在敲过的命令之间来回切换，找到想再跑一次的那条，按回车。刚上手时，很多操作就是在上一条的基础上改一两个词。

这两样都是 shell 提供的，不是某个命令的功能。bash 默认就带着它们，所以从第一次打开终端起，你就能用。

不认识的东西，去哪儿问

命令行的命令成千上万，没人记得住。好消息是不用记住——几乎每个程序都自带说明书。

短的那份，是在程序后面加 `--help`：

```
$ ls --help
```

很多程序都实现了它，输出一段简短的用法摘要，够你确认某个选项怎么拼。但它是程序自己实现的约定，不是 shell 的规矩，所以偶尔也会遇到不认这个参数的程序。

完整的那份是手册页（manual page），用 `man` 打开：

```
$ man ls
```

手册页会把每个选项的意思逐条列出，后面通常还有示例和注意事项。读的时候，空格往下翻，`q` 退出，输入 `/` 再跟关键词可以在页内搜索。

手册页分成若干节：第 1 节是用户命令，第 5 节是文件格式，第 8 节是系统管理命令。同一个名字出现在不同节时，可以在名字前面写节号指定，比如 `man 5 passwd` 查的是密码文件格式，而不是改密码的那个命令。

这两条路的区别很简单：`--help` 快，够用就停；`man` 全，要弄清细节的时候再看。

到此为止，你已经知道一条命令由哪几部分组成、结果会流向哪里，也知道了该去哪儿查你不会的东西。真正让人卡住的从来不是记不住命令，而是不知道该去哪里找答案——这个问题的答案，现在你有了。

文件系统层次

路径即约定



记住这九个位置，能读懂绝大多数 Arch Wiki 文档里的路径

文件系统层次标准（FHS）中最常用的九个位置

第 3 章

文件系统与权限

一切皆文件

Linux 里有一句话几乎能解释所有奇怪现象：一切皆文件（everything is a file）。它的意思不是"所有东西都是文本"，而是说系统给各种资源套上了同一层接口——打开、读取、写入、关闭。

普通文档是文件；目录是文件；硬盘是文件；键盘是文件；正在运行的进程的状态也可以是文件。比如 `/dev/sda` 代表第一块硬盘，往它上面写字节就是往硬盘上写字节；`/proc/cpuinfo` 里读到的是一份"文件"，其实是内核在你读取时现场生成的 CPU 信息。它们不占多少磁盘空间，却都能用同样的 `cat`、同样的重定向 `>` 去操作。

这层统一的代价是抽象：你必须先知道"这个文件代表什么"，才能理解对它操作会发生什么。好处是，几乎所有的工具都只需要会操作文件，就能操作整个系统。

目录树与 FHS

文件系统（file system）是把存储空间组织成文件与目录的规则。Linux 把所有这些目录串在一棵树上，树根是 `/`，也就是根目录。没有 C 盘 D 盘，只有 `/` 和它下面的分支。一条路径从 `/` 写起叫绝对路径（absolute path），从当前目录写起叫相对路径（relative path）；`.` 表示当前目录，`..` 表示上一级。

树的形状不是内核随意决定的，而是遵循一份约定，叫文件系统层次结构标准（Filesystem Hierarchy Standard，FHS）。先记住主干即可：

```
/            根，一切从这里开始
├── etc      系统和软件的配置文件
├── home     普通用户的家目录，形如 /home/用户名
├── root     root 用户的家目录（它不在 /home 下）
├── usr      系统和软件安装的内容
├── var      会变的数据：日志、缓存、数据库
├── tmp      临时文件，重启后不保证还在
├── dev      设备文件
├── proc     内核暴露的进程与系统信息
├── sys      内核暴露的设备与驱动信息
├── boot     内核和引导相关文件
├── mnt      手动挂载用的位置
└── opt      额外安装的软件
```

看到 Arch Wiki 上写"编辑 `/etc/xxx.conf`"或"日志在 `/var/log`"，指的就是这棵树里的固定位置。理解 FHS 的价值在于：你能大概猜到一个陌生文件属于哪一类，也能判断一条教程里的路径是否合理。

读懂 ls -l 的每一列

`ls` 只给名字，加 `-l` 才给细节。典型的输出长这样（示例，名字是虚构的）：

```
$ ls -l
-rw-r--r-- 1 alice users 4200 Mar  8 10:12 notes.txt
drwxr-xr-x 2 alice users 4096 Mar  8 09:30 docs
lrwxrwxrwx 1 root  root     7 Mar  1 08:00 lib -> usr/lib
```

从左往右，一共七列。

第一列的前十个字符是重点，先拆成"类型 + 九位权限"。第一位是文件类型：`-` 普通文件，`d` 目录，`l` 软链接，`c` 字符设备，`b` 块设备，`p` 命名管道，`s` 套接字。随后的九个字符切成三组，分别是属主（owner，也叫 user）、属组（group）、其他人（others）的权限，每组三位，依次是 `r`、`w`、`x`，没有该权限就显示 `-`。

- `r`（read，读）：对文件是能读出内容；对目录是能列出里面有哪些名字。
- `w`（write，写）：对文件是能改内容；对目录是能在里面创建、删除、重命名文件。
- `x`（execute，执行）：对文件是能当作程序运行；对目录是能进入并穿过它访问里面的东西。

目录的 `r` 和 `x` 必须分开理解，这是新手最容易卡住的地方。只有 `r` 没有 `x`，你能看到目录里有哪些文件名，却打不开其中任何一个；只有 `x` 没有 `r`，你进得去、能访问已知名字的文件，但列不出目录内容。

第二列是硬链接（hard link）计数，下文细说。第三、四列是属主和属组。第五列是大小，普通文件以字节计；设备文件这一列显示的是主设备号和次设备号，不代表存储容量。第六列是最后修改时间。第七列是名字；如果是软链接，还会跟一个 `->` 和它指向的目标。

`ls -l` 后面不加参数时列的是目录里的内容。想看目录本身的信息要用 `ls -ld 目录名`，否则你看到的是目录里面，而不是目录这个"文件"本身。

权限怎么算，怎么改

`r`、`w`、`x` 各对应一个数值：`r` 是 4，`w` 是 2，`x` 是 1。三位一组求和，就得到一位八进制数字。于是三组正好三位数：

- `rwxr-xr-x` → 755

- `rw-r--r--` → 644
- `rwx------` → 700

改写权限用 `chmod`（change mode）。数字写法直接给三位数：

```
$ chmod 644 notes.txt
$ chmod 755 script.sh
```

符号写法更直观，`u` 代表属主、`g` 代表属组、`o` 代表其他人、`a` 代表全部，`+` 加权限、`-` 减权限、`=` 直接设成：

```
$ chmod u+x script.sh
$ chmod go-w notes.txt
```

新建文件的初始权限不是凭空来的。系统先给出一个"上限"，文件通常是 666，目录通常是 777，再按掩码（umask）把某些位清掉。常见的 umask 是 022，于是普通文件落在 644，目录落在 755。注意这是按位清除，不是算术减法。

属主和属组用 `chown`（change owner）改，属组也可以用 `chgrp`：

```
# chown alice notes.txt
# chown alice:users notes.txt
```

这两条前面用 `#`，因为改属主属于特权操作，普通用户只能把自己的文件转给别人名下这一种情况几乎不成立，实践中都要 root。`-R` 可以递归处理整个目录，但请务必确认路径再按回车——递归改错属主会让系统文件出问题。

还有两个特殊的权限位值得认识。一个是粘滞位（sticky bit），在权限串里显示为 `t`，比如 `/tmp` 是 `drwxrwxrwt`：谁都往里写，但只能删除自己的文件，删不了别人的。另一个是设置用户 ID（setuid），显示为属主执行位上的 `s`。像 `/usr/bin/passwd` 是 `-rwsr-xr-x`，普通用户执行它时，进程会临时以文件属主（这里是 root）的身份运行，这正是"普通用户也能改自己的密码"的原因。

顺带说一句 root 和权限的关系：root 基本可以绕过读和写的检查，但想执行一个文件，仍然需要它至少有一个 `x` 位。所以"root 什么都行"也不完全对。

硬链接与软链接

要理解链接，先要理解索引节点（inode）。文件系统里真正记录"数据在磁盘的哪里、多大、谁拥有、什么时候改过"的结构是 inode，文件名只是贴在这个 inode 上的一个标签。目录的本质就是一张"名字 → inode 编号"的表。

硬链接（hard link）就是给同一个 inode 再贴一个标签。用 `ln 原文件 新名字` 创建。两个名字完全平等，没有谁是"原件"，删掉其中一个，另一个照样能打开；只有当链接计数归零，数据才真正被回收。`ls -l` 的第二列就是这个计数，新建的普通文件是 1，每多一个硬链接就加 1。目录的这个数字含义稍有不同：每个目录自带一个 `.`

指向自己，再加上它在上层目录里的那个名字，起点就是 2，之后每多一个子目录再加 1。

硬链接有两条硬限制：不能跨文件系统，因为 inode 编号只在单个文件系统内唯一；一般也不允许给目录建硬链接，否则容易造出循环的目录树。

软链接（symbolic link，也叫符号链接）是另一种东西：它本身是一个独立的小文件，内容是"我要指向的那个路径"。用 `ln -s 目标 链接名` 创建。它能跨文件系统，能指向目录，也能指向一个根本不存在的位置——这种叫悬空链接（dangling symlink），访问时会报"No such file or directory"。

两者的差别可以用 `ls -i` 看 inode 验证：硬链接的两个名字 inode 相同，软链接的 inode 不同。软链接多一次跳转，代价是目标一旦改名或挪走它就失效；硬链接没有这个风险，但要占用额外的目录条目且受文件系统限制。

Arch 系统里 `/bin`、`/sbin`、`/lib` 都是指向 `/usr` 下对应目录的软链接。这不是凑巧，而是"usr 合并"（usr merge）的结果：历史上这些目录是分开的，现在统一收进 `/usr`，旧路径保留为软链接，老教程里的 `/bin/ls` 依然能用。这也是为什么读老资料时看到 `/bin` 不要觉得它已经过时。

删除软链接要删链接本身，用 `rm 链接名`，这样不会碰到目标。注意不要在名字后面多打一个斜杠：`rm link/` 会因为要求它必须是个目录而直接报错，什么也删不掉。

挂载：目录树是怎么长出来的

硬盘、分区、U 盘这些存储设备，在格式化之后只是"一块有文件系统的空间"。它不会自动出现在目录树里，需要被"接"上去才看得见。这个动作叫挂载（mount），接上去的那个目录位置叫挂载点（mount point）。挂载完成后，你访问那个目录，看到的就是那块存储的根目录。

```
# mount /dev/sdb1 /mnt
# umount /mnt
```

看当前有哪些块设备、谁挂在哪里，常用 `lsblk`；看各挂载点还剩多少空间，用 `df -h`。

理解挂载最关键的一点是：挂载会"盖住"挂载点目录原本的内容。原来的文件并没有被删除，只是被暂时遮住，卸载之后又会露出来。所以把重要文件写进一个准备当挂载点的目录，是很常见的踩坑方式——挂载之后它们"消失"，卸载之后它们又回来了。

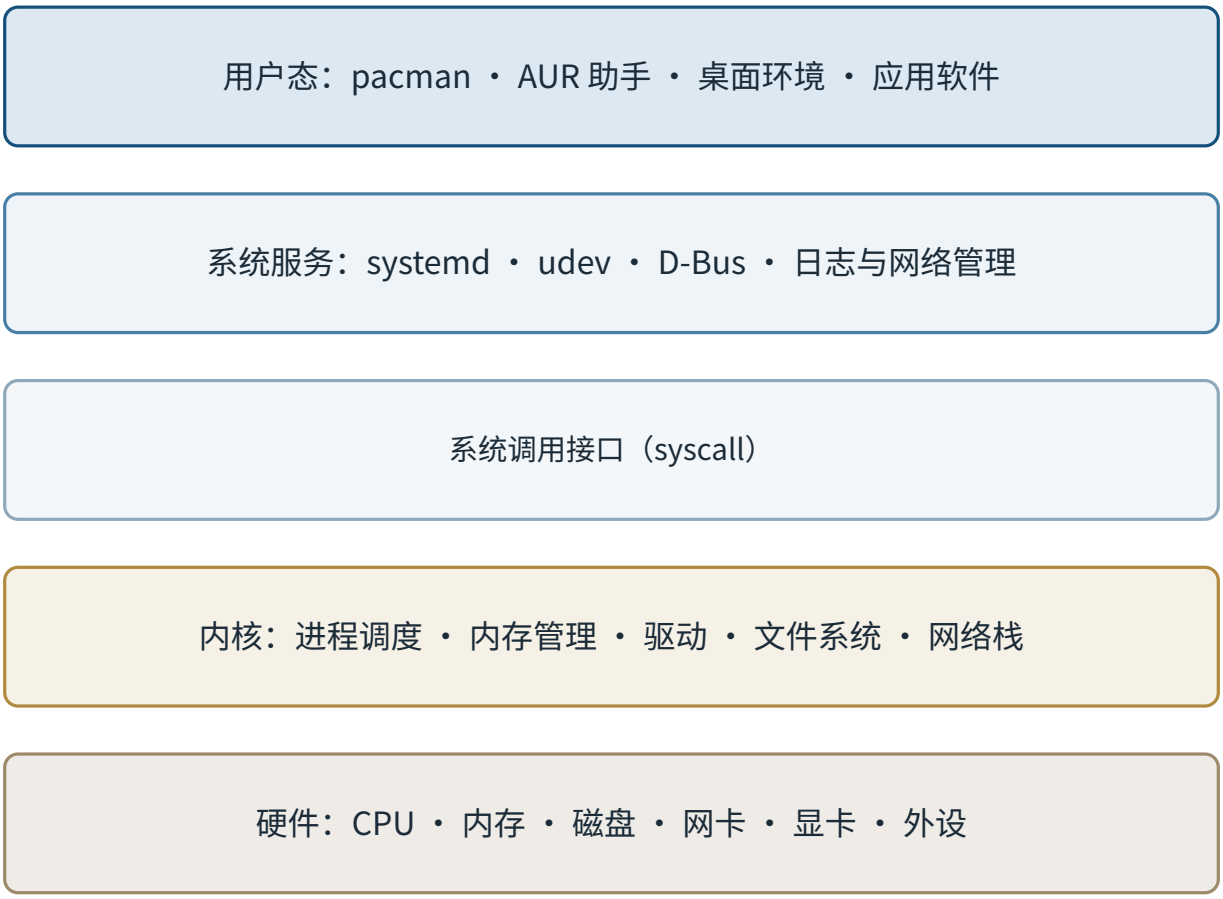
开机时要自动挂载的条目写在 `/etc/fstab` 里，大致是"设备 挂载点 文件系统类型 选项"这样的几列。这个文件写错可能让系统起不来，动手之前先备份，改完用 `mount -a` 之类的方式验证一遍再重启。

`/proc`、`/sys`、`/dev` 这些目录也是在系统启动过程中由内核挂上来的，属于伪文件系统（pseudo filesystem）：里面没有真正的磁盘数据，读到的是内核此时此刻的状态。这也正好回到开头那句话——它们也是文件，只是背后没有硬盘。

权限和挂载这两件事，几乎每一页 Arch Wiki 都会碰到：改一个配置文件的权限、往 `/etc` 里添一个文件、把新分区挂到 `/mnt` 再搬东西进去。把这一章的读法练熟，后面看文档时就不必再猜那些符号是什么意思了。

系统层次

从上到下理解一台 Arch 机器



上层依赖下层提供的接口；越往下越接近硬件，越不适合直接操作

Arch Linux 的五层结构：用户态、系统服务、系统调用、内核与硬件

第 4 章

软件包与依赖

先看看源码安装

大多数开源项目的主页上都挂着一个源码包，通常是压缩后的归档文件。下载、解压，里面的结构大同小异，跟着三步走：

```
$ ./configure
$ make
$ make install
```

第一步叫配置（configure）：检查你的系统里有没有它需要的东西——编译器、某个库、某个头文件，然后把检查结果写成一个构建文件。第二步编译（make）：按构建文件里的规则，把成千上万个源文件翻译成机器能执行的东西，这一步常常要等上几分钟到几十分钟。第三步安装（make install）：把编译出来的程序、库和文档复制到系统目录里。

看起来不难。但装完第一个之后，麻烦才开始。

`make install` 干的事本质上就是复制文件。它不会告诉系统"我装了某某软件"，也不会在任何地方留下清单。于是你很快会遇到几件事：

- 想卸载，没有卸载命令。只能翻回源码目录试试 `make uninstall`，而很多项目根本没写这条规则；就算写了，也只认得当初安装时的那份配置。
- 想升级，得把三步重来一遍。新版本又依赖了新的库，`configure` 会中途停下，告诉你找不到某某。你再去为那个库重复上面所有步骤，一层套一层。
- 想知道某个文件是谁放进来的，说不清。它和你以前装的软件混在同一个目录里。
- 想知道自己到底装了哪些东西，同样说不清。

这就是依赖地狱（dependency hell）的雏形：软件越来越多，彼此要求的版本还会互相矛盾，而系统里没有任何一个角色掌握全局信息。谁装的、谁依赖谁、能不能删，全靠你的记忆。

包管理器在管什么

包管理器（package manager）首先要解决的就是"没人记账"这件事。

它的第一个动作，是把"软件"变成一个边界清楚的东西，叫软件包（package）：一个归档文件，里面装着编译好的程序和库，外加一份描述信息（metadata），写明这个

包叫什么、版本是多少、装了哪些文件、依赖哪些别的包。你在源码安装里手打的那三步，打包者已经在自己的环境里替你做完了。

有了这个边界，包管理器才能做四件事：

- **安装与卸载**。卸载时按清单把文件逐个删掉，不多删也不漏删。
- **记账**。装了什么、装的是哪个版本、某个文件属于哪个包，都写进一份本地记录。
- **解析依赖**。装一个包之前，先算出它还需要哪些包，一次性备齐。
- **统一升级**。所有包按同一份仓库信息一起推进，让彼此的版本保持对得上。

包里的文件也不是随便丢进系统，而是按第三章讲的目录习惯放在固定位置：程序进 `/usr/bin`，库进 `/usr/lib`，配置文件进 `/etc`。正因如此，一个包在系统里留下的痕迹是可枚举的——这是记账能成立的前提。

仓库：别人替你编译好的地方

仓库（repository，常简称 repo）是一台服务器上的一组软件包，由发行版维护。Arch 的官方仓库按用途分成几个，各自装着一批由官方打包者维护、签名过的包。

从用户角度看，仓库省掉的是编译。从发行版角度看，仓库真正提供的是**一致性**：这些包是在一套约定的环境里、按同一批依赖版本编译出来的，装到一起就能跑。你自己编译的那份二进制，落在这个约定之外。

仓库还有个容易被忽略的作用：集中信任。每个包由打包者签名，你的系统只接受持有对应密钥的人打出来的包。所以"从仓库装"这个动作包含的不只是下载，还有一次签名校验。

数据库：系统对自己知道多少

包管理器把账记在两个地方，理解这两个地方的区别，后面很多现象就顺了。

一处是**本地数据库**（local database），记的是"这台机器上实际装着什么"，位于 `/var/lib/pacman`。你问系统"这个文件是谁装的""我是不是装过某个包""这个包依赖什么"，读的都是它。

```
$ pacman -Qo /usr/bin/vim
$ pacman -Qi vim
```

第一行问的是某个文件属于哪个包，第二行问某个已安装包的信息。这类查询之所以能立刻回答，正是因为安装时写下了账。

另一处是**同步数据库**（sync database），是仓库索引在你机器上的一份副本。它记的是"仓库里有什么、什么版本、依赖谁"，不表示你已经装了。搜索和查询仓库里的包读的是它：

```
$ pacman -Ss 关键词          # 关键词替换成你要找的东西
```

同步数据库是从仓库下载来的，会随着你每次同步被替换。它只是一份目录，不是文件本身。这个区别很重要，下面讲滚动更新时会用到。

依赖解析：一张不断变动的图

把包和依赖画成图，每个包是一个点，依赖是一条有向的边。装一个包，就是要把以它为起点的整片可达区域都找出来。

难点在于这张图不是静态的。同一份依赖可能同时被好几个包需要，各方的版本要求还可能互相冲突；有的包能顶替另一个包的角色（提供同样的能力，但实现不同）；有的包之间则不能共存于同一套文件布局之下。所以依赖解析不是"查一张表"，而是一次求解：在满足全部约束的前提下，挑出一组能同时装上的版本。

这也是为什么包管理器报错时，说的往往不是"找不到包"，而是"无法满足依赖"或"某两个包冲突"。它在告诉你：这件事在你的系统当前状态下无解，需要你先做取舍。

滚动更新：仓库是一个整体

第一章提过 Arch 采用滚动更新（rolling release），这里要补上它对包管理器提出的额外要求。

在固定发布的发行版里，一次大版本发布就是一个被冻结的包集合，所有包的依赖都在这个集合内部对齐。Arch 没有这样的集合——仓库在持续被替换，任何时刻它都代表"当前最新的整体状态"。于是有了一条推论：

仓库里所有包的依赖关系，是按整份仓库对齐的。 只要你同步了仓库索引，就应该把所有已装软件一起升到这份索引对应的状态。只升其中一部分，就等于把系统的一半锁在旧状态、一半推到新状态，而这两半之间的接口并没有人保证。

下面的写法就是典型的错误操作：

```
# pacman -Sy 某个包
```

`-Sy` 只刷新索引，不带升级。它的结果是：你的索引已经很新，已安装的软件还是旧的，接着再装一个新包，新旧两套依赖就被拼到一起了。Arch 把这种情况叫部分更新（partial upgrade），是官方明确劝阻的。要装东西，就把"同步 + 全体升级"当成一个不可拆开的动作：

```
# pacman -Syu
```

滚动模型的另一面是：更新是日常动作，不是年度事件。长期不同步之后一次性追上，中间跨越的状态越多，出问题的概率越高。官方偶尔也会要求人工介入——比如某个包的配置文件格式变了，需要你手动处理。这些都会写进更新公告里。

AUR：一份配方，不是一台仓库

Arch 用户仓库（Arch User Repository，AUR）经常被误解。名字里虽然有 repository，它和官方仓库不是一类东西。

官方仓库里放的是**编译好的包**。AUR 里放的是**构建配方**：一份叫 PKGBUILD 的脚本，加上配套文件，说明从哪个上游地址取源码、怎么编译、装到哪儿。AUR 不提供二进制文件，也没有人替你在服务器上编译。

所以"从 AUR 装东西"实际是两步：取回配方，在你的机器上现场构建，再装成品。社区写的若干助手工具（例如 `yay`、`paru`）只是把这两步串起来做得顺手些，它们不是官方组件，Arch 官方也不为它们背书。

AUR 的价值很直接：官方仓库不可能收录所有软件，尤其是一些小众工具、上游闭源的东西、或打包者不认为该进官方仓库的软件。它把"打包"这件事开放给了社区。

风险同样直接，而且有几层：

- **没有人在替你把关成品。** 官方仓库的包有打包者签名，AUR 的配方只有作者和投票记录。配方里能写任意命令，构建过程在你的机器上以你的身份运行。所以装之前看一眼 PKGBUILD，是个应有的习惯，而不是多疑。
- **配方会过期。** 上游换了构建系统、改了依赖，配方没跟上，构建就会失败。这种失败和你的系统没关系，是配方本身旧了。
- **版本节奏不由官方约束。** AUR 包通常跟着上游走，而库的升级会让旧二进制失去兼容。滚动更新之后，AUR 里装的东西常常需要重新构建一次才能继续用。
- **可能撞车。** AUR 包和官方包装了同一种东西，就可能互相覆盖文件或产生冲突。

把这些放在一起，AUR 的定位其实很清楚：它是官方仓库的补充，不是官方仓库的替代，也不在支持的范围内。能用官方包解决的，用官方包。

pacman 的取舍

`pacman` 是 Arch 的包管理器。回头看，它的性格和第一章讲的 Arch 之道是一致的：只提供机制，不替你下判断。

具体表现为几条一以贯之的选择：

- **只做被要求的事。** 它不弹窗、不自动确认、不猜你的意图。要装就装，要删就删，不问第二遍。
- **记账清楚，但不代为解释。** 装了哪些文件、谁依赖谁，全都能查；但它不会主动告诉你"这个操作可能影响其他包"。
- **不自动清理依赖。** 删一个包，默认只删它自己。它顺带用到的库会留在系统里——因为那些库可能还有别的东西在用，判断权交给你。想连不再需要的依赖一起删，得明确说出来。
- **不替你合并配置。** 更新时如果你的 `/etc` 下某个文件被改过，它会保留你的版本，另外放一份新版本在旁边，由你决定怎么合。
- **不在出错时兜底。** 下载过的包会留在缓存目录里，所以回退到旧版本是可能的；但回退这个决定，仍然要你自己做。

这套取舍的代价，就是你得知道自己敲的命令意味着什么。好处是，系统里几乎不存在"不知道为什么变成这样"的地方——包这一层尤其清楚：装了什么、从哪儿来、被谁依赖，全都有账可查。

第二册会把这本账翻开来细看：数据库里到底记了什么、依赖冲突时怎么读输出、AUR 的配方怎么写、系统滚坏了怎么退回去。前提是这一章的东西已经不再陌生——包是被管理起来的文件集合，仓库是别人替你编译好的一致集合，更新是一个整体动作而不是零散操作。

第 5 章

系统与服务

开机之后，谁在干活

按下电源键以后，主板固件把控制权交给引导程序，引导程序加载内核。内核做完自己的事——初始化硬件、挂载根文件系统——就会启动它找到的第一个用户空间程序，并把它的进程号（PID）定为 1。

这个数字很要紧。内核本身不管服务，也不管网络和登录，它只负责把第一个进程拉起来。此后系统上所有"不用人盯着也一直在跑"的东西，都是这个进程及其子孙在承担。这个角色叫作 init，取自 initialization（初始化）。

历史上各家发行版用过不同的 init。Arch 早期用的是 SysV init，后来换成了 systemd。systemd 自己就是那个 PID 1，同时兼作服务管理器。想知道当前系统上是谁在当 PID 1，可以看进程表：

```
$ ps -p 1 -o comm=
systemd
```

顺带记一个词：后台常驻、不需要人交互就能自己运行的程序，叫作守护进程（daemon）。systemd 的核心职责就是把这类守护进程按时拉起来、看着它们、必要时关掉。

单位：systemd 眼里的万物

systemd 不把系统想成一堆启动脚本，而是想成一堆"单位"。单位（unit）是它管理的最小对象，每个单位由一个配置文件描述，文件名后缀说明它属于哪种类型：

- `.service`：一个后台服务，最常见的一种
- `.socket`：一个被监听的端口或套接字，有人连上来时才唤醒对应服务
- `.timer`：定时触发器，作用类似传统的 cron
- `.mount` 与 `.automount`：挂载点
- `.path`：监视某个文件或目录的变化，变了就触发动作
- `.target`：一批单位的集合，下一节再说

单位文件主要来自两个地方：`/usr/lib/systemd/system/` 放软件包自带的，升级时会被覆盖；`/etc/systemd/system/` 放管理员自己写的，优先级更高，同名会盖住前者。这种"系统目录加本地目录、后者覆盖前者"的安排在 Arch 里反复出现，值得先记住。

也正因为如此，想改某个单位的行为，不要去动 /usr/lib 下面的文件，正确做法是在 /etc/systemd/system/ 里放一个同名文件，或者建一个形如 名称.service.d/ 的目录，把要改的几行写进去。后者叫 drop-in，注意力集中、不容易和升级冲突。systemctl edit 名称.service 会自动帮你建好这个目录并打开编辑器，systemctl cat 名称.service 则把上层文件和所有 drop-in 拼在一起显示出来，改之前先看一眼它到底长什么样。

有时候某个服务被别的单位当依赖顺手拉起来，你并不想让它跑。这时 # systemctl mask 名称.service 把它彻底封掉——它会被链接到 /dev/null，即便有人想启动它也无从下手；解除用 systemctl unmask。比起 disable，mask 是更硬的封锁。

目标：一批单位被打包

系统的启动过程可以拆成阶段：先是文件系统可用，然后网络就绪，最后才轮到图形界面。systemd 用目标（target）来表示"系统现在应该处在哪个状态"。

target 自己几乎不干活，它更像一个标签或者一个组。启动一个 target，等于启动它名下的一批单位。常见的几个：

- multi-user.target：多用户、有网络，但没有图形界面
- graphical.target：在 multi-user.target 的基础上再拉起显示管理器
- rescue.target：救援模式，只起最基本的服务
- poweroff.target 与 reboot.target：关机与重启

老 SysV 时代有个"运行级别"（runlevel）的概念，现在基本被 target 取代，systemd 保留了一点兼容映射。默认要进哪个 target，可以查也可以改：

```
$ systemctl get-default
graphical.target
# systemctl set-default multi-user.target
```

开机时 systemd 直奔默认 target，顺带拉起整条依赖链。所谓"服务开机自启"，机制就在这里。

启停与开机自启

日常打交道最多的就是这几个动作，先把它们分清：

- start / stop：现在启动、现在停止
- restart：停掉再起来；reload：不中断服务，只让它重读配置
- disable / enable：下次开机是否自动启动
- status：看它现在怎么样

最容易被弄混的是 `start` 和 `enable`。`start` 只影响眼下这一次运行，重启机器就没了；`enable` 只影响下次开机，不会立刻把服务跑起来——除非加上 `--now`。

```
# systemctl status 名称.service
# systemctl start 名称.service
# systemctl restart 名称.service
# systemctl enable 名称.service
# systemctl enable --now 名称.service
# systemctl disable 名称.service
```

`status` 的输出值得逐行看明白。Loaded 一行说明单位文件从哪儿来、是否处于 enabled 状态；Active 一行是关键，`active (running)` 表示正在跑，`inactive (dead)` 表示没在跑，`failed` 表示启动失败了。Active 下面的几行是最近几条日志，通常够判断问题出在哪，末尾还会提示你用 journalctl 去看更多。

依赖关系和排错时常用的几条：

```
$ systemctl list-units --type=service      # 内存中已有的服务
$ systemctl list-unit-files --type=service # 磁盘上有哪些，各自是否自启
$ systemctl --failed                       # 当前失败的单位
$ systemctl list-dependencies 名称.service # 它依赖谁
```

服务起不来时，先别急着重启一遍，用 `status` 和日志看清原因，往往比反复重试省时间。

日志在哪里

systemd 自带一个日志服务 systemd-journald，它收集内核消息、各个服务的标准输出与错误输出，以及从别处转来的日志。查询工具是 `journalctl`。

日志落在哪儿分两种情况。如果 `/var/log/journal/` 这个目录存在，日志会写到磁盘，重启后还在；如果它不存在，日志就只写到内存里的 `/run/log/journal/`，关机即失。Arch 默认不建这个目录，所以想要跨重启保留日志，自己创建一次即可：

```
# mkdir -p /var/log/journal
# systemctl restart systemd-journald
```

常用查询：

```
$ journalctl                # 全部日志，从旧到新
$ journalctl -b             # 只看本次启动
$ journalctl -b -1          # 上一次启动
$ journalctl -u 名称.service # 只看某个服务
$ journalctl -f             # 持续跟踪新日志
$ journalctl -k             # 只看内核消息
$ journalctl -p err -b      # 本次启动中优先级为 err 及以上
$ sudo journalctl --disk-usage   # 日志占了多少磁盘
```

`--since` 与 `--until` 可以按时间切片，写法比较宽容，`"1 hour ago"`、`"yesterday"`、`"2024-01-01 10:00:00"` 都认。把这些选项组合起来，

比如 `journalctl -u 名称.service -b --since "10 min ago"`，就是排查"刚才那个服务为什么挂了"的标准动作。

日志有上限，不会无限长大，默认会在磁盘占用过多时丢掉最旧的部分。确实需要腾空间时，可以手动清到某个尺寸：`# journalctl --vacuum-size=500M`。

权限方面，普通用户默认只能看到自己的日志。要读全系统的日志，需要是 root，或者把自己加进 `systemd-journal` 组：

```
# usermod -aG systemd-journal 用户名
```

除了 journal，`/var/log/` 下还有一些由程序自己写的纯文本日志。其中 `pacman.log` 记录了软件包安装、升级、卸载的全部历史，想知道"这个包是哪天装进来的"翻它就行。

用户、组与 sudo

系统里每个进程都带着一个用户身份在跑。用户用 UID 标识，root 的 UID 固定是 0，拥有绕过一切权限检查的能力；普通用户的 UID 一般从 1000 往上排。

用户信息分散在三个文件里：`/etc/passwd` 记录用户名、UID、主组、家目录和登录 shell，任何用户可读；`/etc/shadow` 存密码哈希，只有 root 能读；`/etc/group` 记录组和组成员。

组（group）是把权限打包分发的手段——进了某个组，就继承这个组拥有的权限。每个用户有一个主组（primary group），此外还可以属于任意多个附加组（supplementary group）。加入 `wheel` 组之所以能让人用 sudo，原因就在这儿。

查看与增删：

```
$ id                        # 自己的 UID、主组、附加组
$ groups                    # 自己属于哪些组
# useradd -m -G wheel 用户名      # 建用户，-m 才会创建家目录
# passwd 用户名                   # 设置密码
# usermod -aG 组名 用户名         # 追加组
# userdel -r 用户名               # 删用户并一并删掉家目录
```

`usermod -aG` 里的 `-a` 不能省。少了它，追加就变成了覆盖，用户原有的附加组会被整条替换掉，很容易把刚配好的权限弄丢。

sudo 解决的是"临时借 root 权限做一件事"的问题。它要求输入的是自己的密码，而不是 root 密码；相对的，`su -` 是切换到 root 并需要一个完整的 root 会话，要输 root 的密码。日常操作更推荐前者，因为它只给一条命令提权，事后自动收回。

sudo 的规则写在 `/etc/sudoers` 里。编辑它必须用 `visudo`，因为 visudo 会在保存前做语法检查——直接编辑留下语法错误，可能让 sudo 彻底不可用。更稳妥的做法是不动主文件，而在 `/etc/sudoers.d/` 下新建一个单独文件：

```
# visudo -f /etc/sudoers.d/10-wheel
```

文件里允许 wheel 组做任何事的那一行长这样：

```
%wheel ALL=(ALL:ALL) ALL
```

开头的 `%` 表示这是组名，不是用户名。这一行的读法是：wheel 组里的所有人，在任何主机上，可以以任意用户身份运行任意命令。Arch 的默认 sudoers 里这条规则是注释掉的，需要自己取消注释，或者写进 `sudoers.d` 里。

配置生效以后，`sudo -l` 会列出你当前被允许执行的命令，改完规则不确定有没有生效，用它可以自查。偶尔需要连着做几件需要 root 的事，`sudo -i` 会给你一个临时的 root 登录 shell，退出即回到自己——比在每条命令前面加 sudo 省事，但仍要记得及时 `exit`。

读懂这些输出

现在再打开一个终端，`systemctl status` 的输出不再是黑箱：Loaded 告诉你单位文件从哪来、是否开机自启，Active 告诉你它现在跑没跑，下面几行是它最近的日志。`journalctl -u` 和 `journalctl -b` 则把时间轴拉出来，让你看到某个服务在某个时刻具体说了什么。

把这条链记住，剩下的都是查：内核拉起 PID 1，PID 1 是 systemd，systemd 按 target 组织起一批单位，每个单位背后是一个守护进程；这些进程的输出统一汇进 journal；而谁能对它们做什么，由用户、组和 sudo 规则决定。这一套东西本身不复杂，只是名字多。名字认全了，后台就不神秘了。

第 6 章

学习方法与故障排查

一、先把 Arch Wiki 当成手册，而不是教程

Arch Wiki（wiki.archlinux.org）不是一本从头读到尾的书，而是一部可以按需查阅的手册。理解它的组织方式，比记住任何一条命令都重要。

它的基本单位是**页面（page）**。每个页面讲一件事，比如一个软件、一个子系统、一个流程。页面开头通常有一段概述，告诉你这个页面解决什么问题；紧接着是**目录（table of contents）**，列出全部小节。多数页面遵循相似的骨架：安装、配置、使用、故障排查（Troubleshooting）、参见（See also）。看到"Troubleshooting"小节，应该优先跳过去，因为前人踩过的坑通常写在那里。

页面之间靠**分类（category）**和内链连接。页面底部会列出它所属的分类，分类页则是同类页面的集合。正文里的蓝色链接，往往指向你下一步需要读的页面。读到一半发现某个概念不懂，直接点进去，读完再退回来，这是 Wiki 的正常读法。

检索有几个实用技巧。第一，**用英文搜**。英文页面最全、更新最快，中文翻译常常滞后甚至缺失。第二，站内搜索框适合搜主题词（如 `network configuration`），不适合搜长句子。第三，搜报错原文时，用搜索引擎加站点限定，把报错的关键片段放进引号：

```
site:wiki.archlinux.org "failed to mount /boot"
```

第四，页面底部有最后修改时间。如果一段配置说明是很多年前的，要警惕它是否还适用于当前版本。第五，Wiki 不是唯一来源。命令自带的 `--help`、手册页（man page，用 `man 命令名` 查看）、软件自己的上游文档，都是更权威的一手资料。Wiki 帮你定位，一手资料帮你确认。

二、报错信息是给你看的，不是给你怕的

大多数报错遵循同一个结构：**谁**在**做什么**的时候，因为**什么原因**失败了。读报错就是把这几个空填出来。

先看第一行和最后一行。第一行通常点明出错的程序和动作，最后一行通常是结论。中间的大段输出往往只是上下文。特别注意报错里出现的**路径**、**单元名**、**包名**、**设备名**，这些是搜索时最有价值的关键词，比"报错了怎么办"有用得多。

区分标准输出和标准错误。正常结果走标准输出（stdout），错误走标准错误（stderr）。把命令的输出重定向到文件时，两者默认分开，排查时更可靠的做法是直接用日志工具看。

命令执行完，可以用退出码判断成败：

```
$ echo $?
```

`0` 表示成功，非 `0` 表示失败。脚本和自动化里全靠这个值判断。

系统层面的日志交给 journal。systemd 把内核、服务、应用的日志汇总在一起，按启动轮次（boot）分组：

```
$ journalctl -b
$ journalctl -b -p err
$ journalctl -u 服务名 -b
```

第一条看本次启动的全部日志，第二条只看错误级别以上，第三条只看某个服务。日志很长时，用 `-p` 过滤级别，比从头翻要快得多。内核自己打印的信息还可以用 `dmesg` 单独看，硬件相关的报错大多在那里。

搜索报错时，**复制完整的一行，连同前后几行上下文**。只搜半句常常搜不到，或者搜到不相关的答案。如果报错里有你机器特有的信息（主机名、用户名、IP、序列号），搜之前先替换成占位符。

三、分层排查：从最底下往上查

系统的启动是一条链：固件（firmware）找到引导程序（bootloader），引导程序加载内核（kernel），内核挂载初始内存盘（initramfs），initramfs 挂载真正的根文件系统，然后交给 init 系统（systemd）启动各项服务，最后由服务拉起应用。

这条链上任何一环断了，表现都可能是"开不了机"。所以排查要**按层进行，从下往上**，一层确认通过再查上一层。跳层排查是最常见的浪费时间的方式。

下面是一份可以照着执行的顺序：

1. **确认硬件和固件层**。开机有没有自检画面，硬盘是否被识别。进固件设置或从安装介质启动，用 `lsblk` 和 `blkid` 看分区和文件系统是否还在。
2. **确认引导层**。引导菜单能不能出现，选中项指向的内核和 initramfs 是否存在。
3. **确认内核与 initramfs**。内核能不能加载，initramfs 里是否包含根分区所需的驱动。近期升级过内核却忘了重建 initramfs，症状就是卡在这一层。
4. **确认根文件系统**。根分区能不能挂载，`/etc/fstab` 里的条目是否与实际分区一致。
5. **确认 init 系统**。systemd 有没有起来，哪些单元失败了：

```
$ systemctl --failed
$ systemctl status 单元名
```

1. **确认服务层**。依赖网络、数据库、显示管理器的服务，是否因为更底层的服务没起来而连带失败。
2. **最后才是应用层**。单个程序崩溃，前面的层通常都是好的。

每一层都问同一个问题：**最近改了什么**。新装的包、改过的配置文件、升级过的内核，是绝大多数故障的直接原因。时间线比猜测可靠。如果记不清，包管理器的日志和文件的修改时间可以帮你还原。

四、五类常见致命错误

第一类：引导链断裂。 症状是开机直接进固件界面，或者停在引导菜单后黑屏。常见原因是升级内核后没有重新生成 initramfs，或者引导程序的配置文件没有更新。确认方法是从安装介质启动并挂载根分区，检查 `/boot` 下的内核与 initramfs 文件是否成对存在。修复需要进入 chroot 环境，重新生成 initramfs 并更新引导配置。

第二类：根分区写满。 症状是登录后什么都做不了，日志里满是"设备上没有空间"（No space left on device）。日志文件、包缓存、下载目录是常见的增长点。用 `df -h` 看各分区占用，再逐层定位大目录。注意给根分区留出余量，写满之后连日志都写不进去，排查会更困难。

第三类：权限与所有者被误改。 典型场景是在根目录下执行了递归的权限修改，或者在错误的位置改了属主。症状五花八门：`sudo` 报错、服务起不来、图形界面登录失败。这类问题的麻烦在于波及面广，报错本身不指向真正的原因。确认方法是检查关键目录的属主和权限是否符合预期；修复要靠包管理器校验文件完整性，把被改坏的文件恢复回来。

第四类：包管理器状态损坏。 包括部分升级（只升级了某个包而没有整体升级）、数据库被锁、依赖关系断裂。症状是安装或升级时报依赖冲突，或者系统里一半是新版本一半是旧版本。原则很简单：升级永远整体做，不要单独升级某个包。数据库被锁时，先确认没有别的包管理进程在运行，再处理锁文件。

第五类：网络与域名解析不通。 这一类本身不致命，但它会让你失去自救能力，所以必须单独列出来。症状是包管理器连不上镜像站，或者能 ping 通 IP 却打不开域名。先分层确认：网卡有没有拿到地址，网关通不通，域名解析是否正常。把这几个问题分开测，才能知道故障出在链路、路由还是域名解析（DNS）。

五、什么时候该重装

重装系统不是失败，是一次成本决策。判断标准只有一个：**修复的预期时间，是否明显超过重装的预期时间**。

倾向于重装的信号：文件系统损坏到无法修复；同一台机器上同时存在多个互不相关的故障；系统被大量手工修改过，已经没人说得清当前状态；你需要在很短时间内恢复一台能用的机器。

倾向于不重装的信号：故障只影响单个服务或单个程序；问题出在配置而不是系统本身；你还能从安装介质启动并挂载根分区。绝大多数引导问题和服务问题都属于这一类，修比重装快。

决定重装之前，先备份。至少包括：`/home` 下的个人文件、`/etc` 下的配置文件、已安装软件包的清单，以及浏览器和密钥环里那些不方便重建的数据。包清单可以导出成文本，重装后照着装回去：

```
$ pacman -Qqe > pkglist.txt
```

如果系统还能启动，先把清单和配置拷到外部存储。如果已经起不来，就从安装介质挂载分区再拷。备份这一步省不得，重装可以重来，数据不行。